



자료구조 실습

11/26



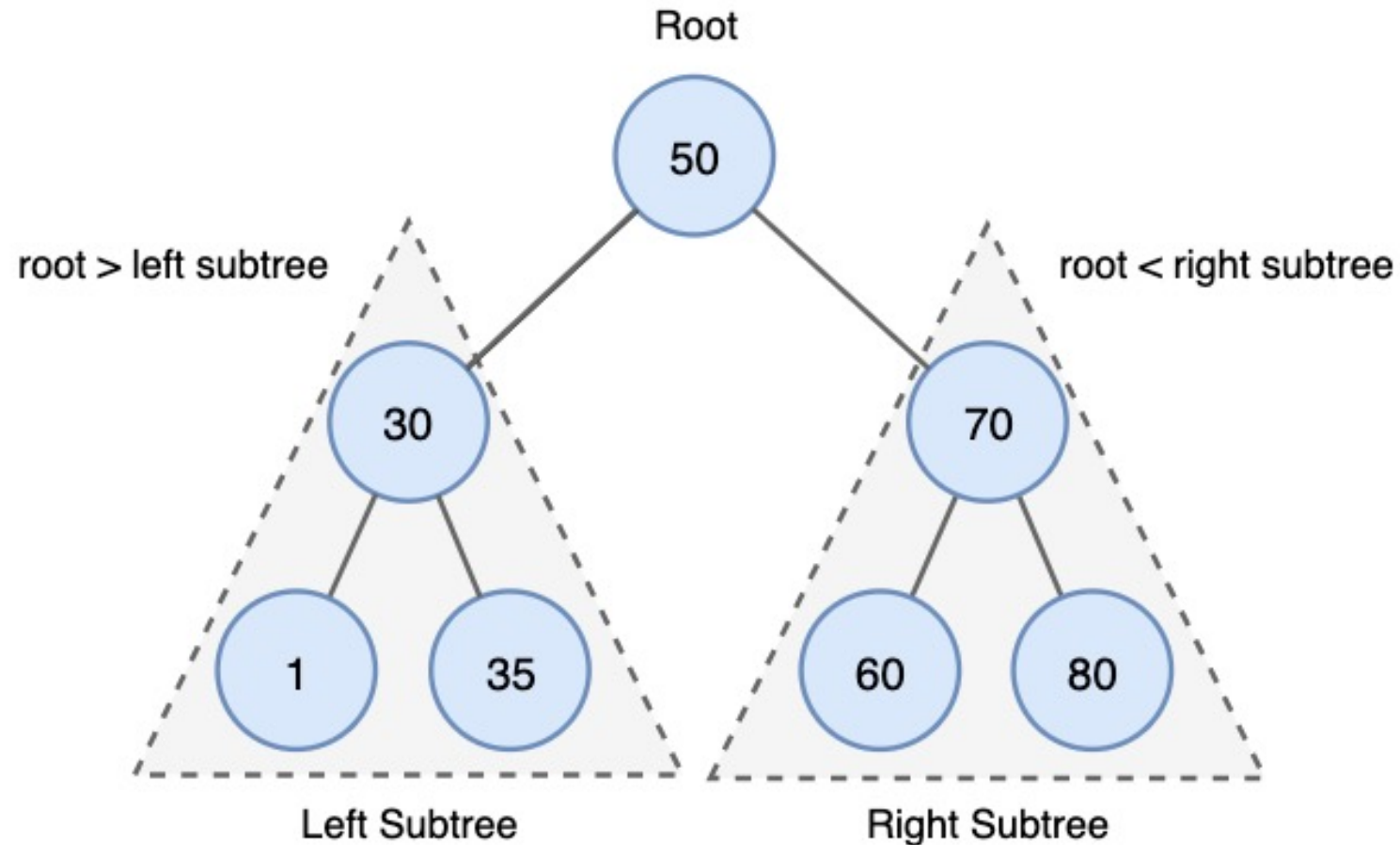


목차

- BST란? (remind)
 - Node, create_node()
- Preorder
- Inorder
- Postorder
- 실행 결과
- 과제 안내



BST (Binary Search Tree) 란?





BST (Binary Search Tree) 란?

```
// 트리의 노드를 표현하는 구조체 정의
struct Node {
    int key;           // 노드가 저장할 값 (정수)
    struct Node *left; // 왼쪽 자식 노드에 대한 포인터
    struct Node *right; // 오른쪽 자식 노드에 대한 포인터
};
```

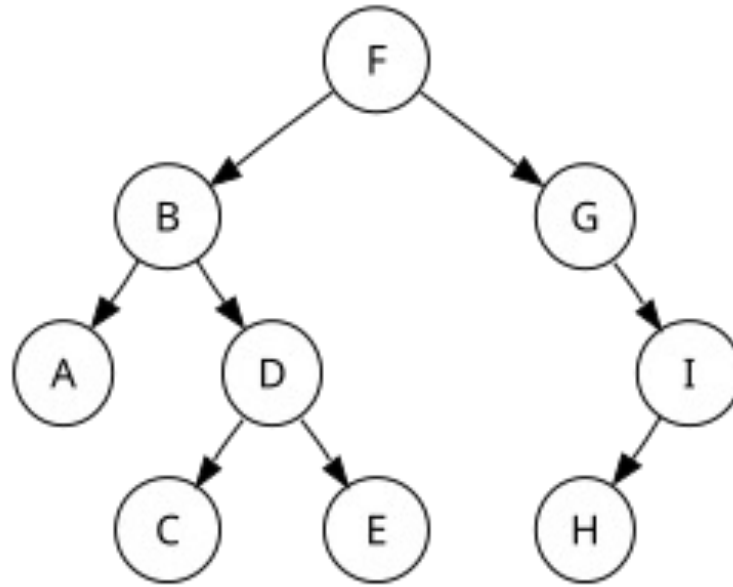


BST (Binary Search Tree) 란?

```
// 새로운 노드를 생성하는 함수 (메모리 할당 포함)
struct Node* create_node(int key) {
    struct Node *node = (struct Node *)malloc(sizeof(struct Node));
    if (node == NULL) { // 메모리 할당 실패 시 확인
        printf("Memory allocation error\n"); // 오류 메시지 출력
        exit(1); // 프로그램 강제 종료
    }
    node->key = key; // 노드의 key 필드에 값 저장
    node->left = NULL; // 초기 상태에서는 자식 노드 없음
    node->right = NULL; // 초기 상태에서는 자식 노드 없음
    return node; // 생성한 노드의 주소를 반환
}
```



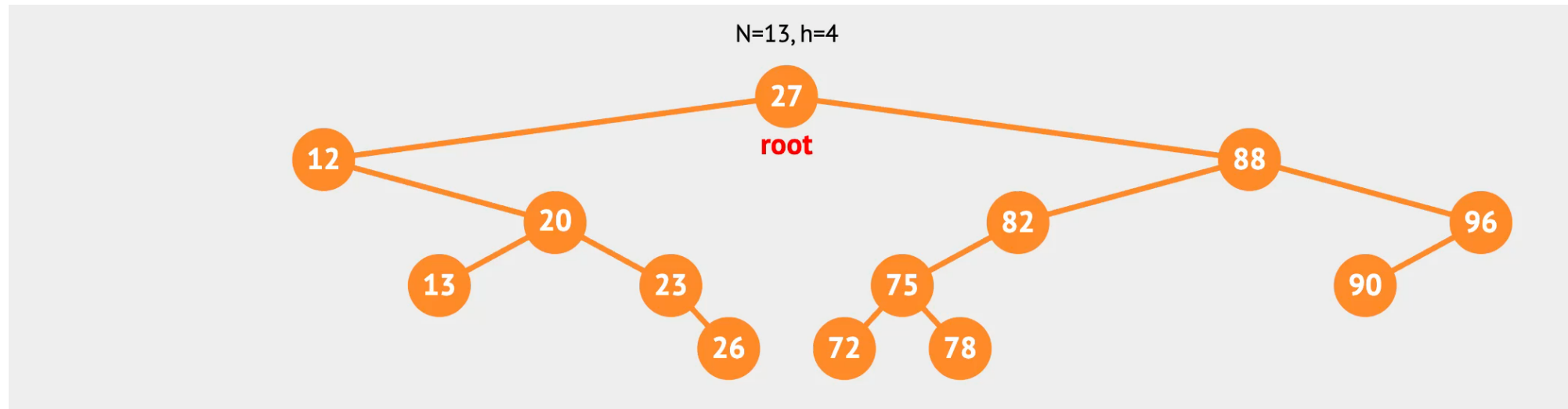
Preorder



Root -> left -> right



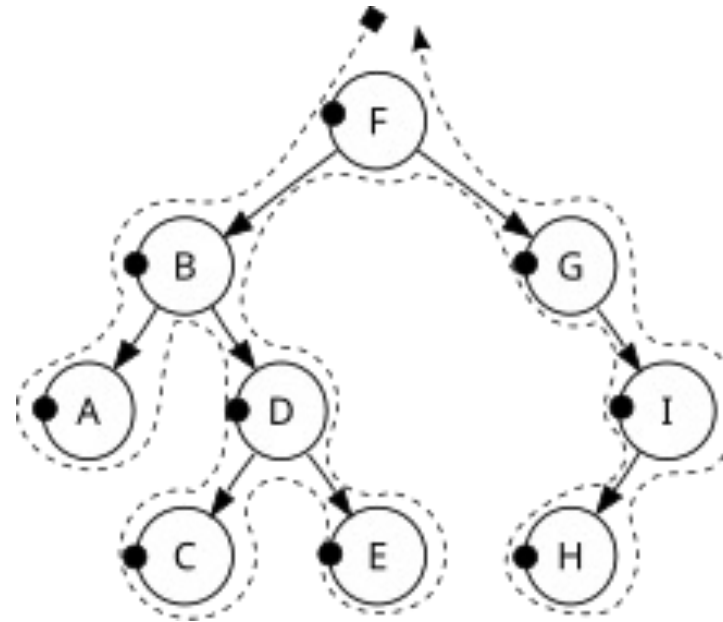
Preorder



Root -> left -> right



Preorder



Root -> left -> right



Preorder

```
// 전위 순회: (현재 노드) → (왼쪽) → (오른쪽)
void preorder(struct Node *root) {
    if (root == NULL) { // 빈 노드(NULL)에 도달하면 즉시 종료
        return;
    }
    printf("%d ", [redacted]); // 현재 노드 방문: key 출력
    preorder([redacted]);      // 왼쪽 서브트리 재귀 호출
    preorder([redacted]);      // 오른쪽 서브트리 재귀 호출
}
```



Preorder

```
// 전위 순회: (현재 노드) → (왼쪽) → (오른쪽)
void preorder(struct Node *root) {
    if (root == NULL) { // 빈 노드(NULL)에 도달하면 즉시 종료
        return;
    }
    printf("%d ", root->key); // 현재 노드 방문: key 출력
    preorder( ); // 왼쪽 서브트리 재귀 호출
    preorder( ); // 오른쪽 서브트리 재귀 호출
}
```



Preorder

```
// 전위 순회: (현재 노드) → (왼쪽) → (오른쪽)
void preorder(struct Node *root) {
    if (root == NULL) { // 빈 노드(NULL)에 도달하면 즉시 종료
        return;
    }
    printf("%d ", root->key); // 현재 노드 방문: key 출력
    preorder(root->left);      // 왼쪽 서브트리 재귀 호출
    preorder( );               // 오른쪽 서브트리 재귀 호출
}
```

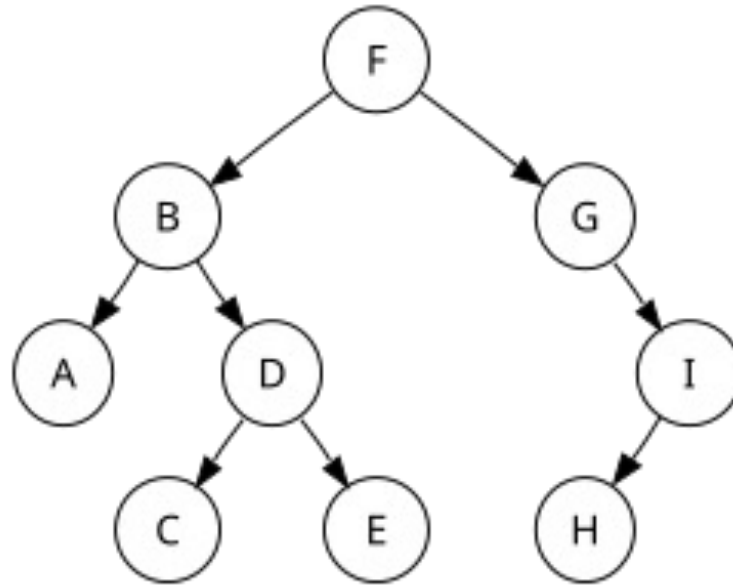


Preorder

```
// 전위 순회: (현재 노드) → (왼쪽) → (오른쪽)
void preorder(struct Node *root) {
    if (root == NULL) { // 빈 노드(NULL)에 도달하면 즉시 종료
        return;
    }
    printf("%d ", root->key); // 현재 노드 방문: key 출력
    preorder(root->left);      // 왼쪽 서브트리 재귀 호출
    preorder(root->right);     // 오른쪽 서브트리 재귀 호출
}
```



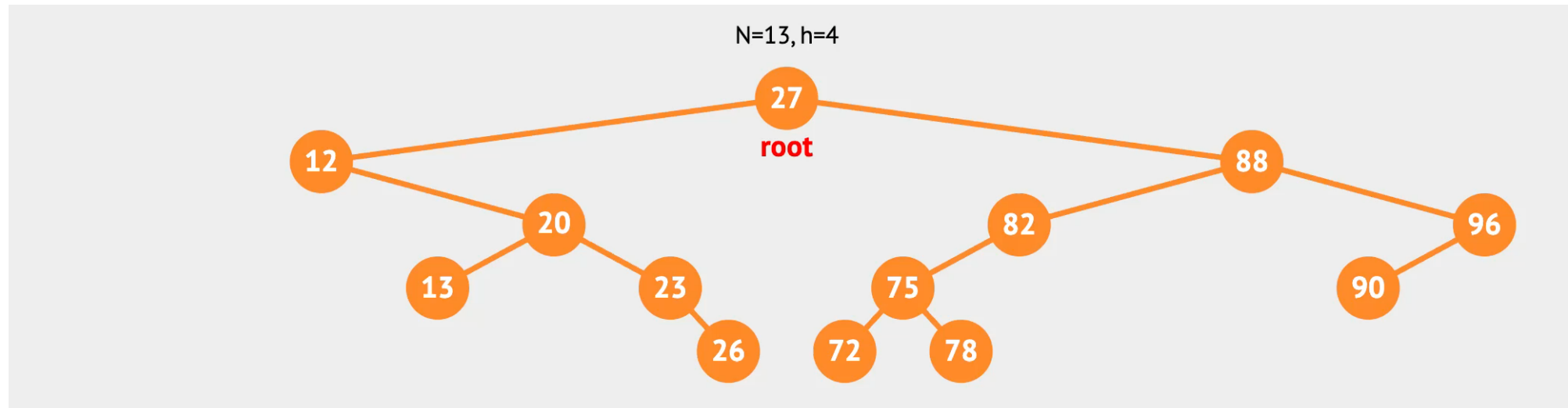
Inorder



left -> root -> right



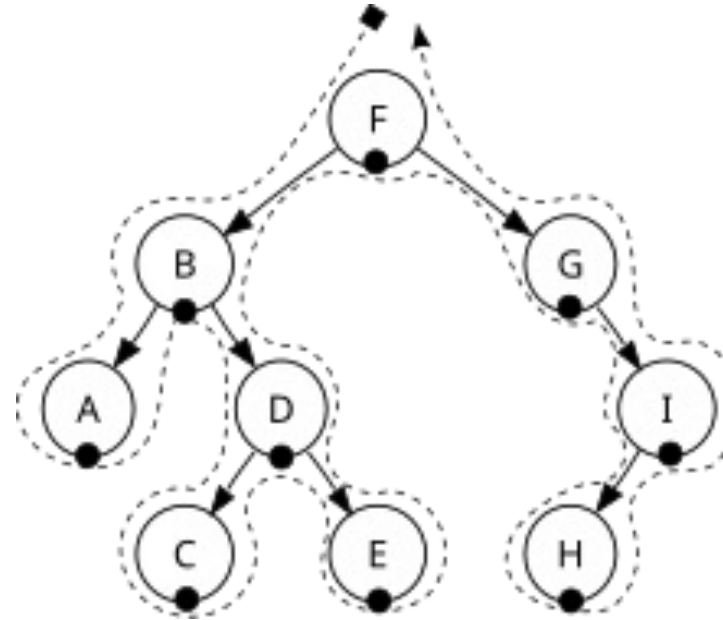
Inorder



left -> root -> right



Inorder



left -> root -> right



Inorder

```
// 중위 순회: (왼쪽) → (현재 노드) → (오른쪽)
void inorder(struct Node *root) {
    if (root == NULL) { // 빈 노드이면 종료
        return;
    }
    inorder( ); // 왼쪽 서브트리를 먼저 방문
    printf("%d ", ); // 현재 노드 방문
    // 오른쪽 서브트리 방문
}
```



Inorder

```
// 중위 순회: (왼쪽) → (현재 노드) → (오른쪽)
void inorder(struct Node *root) {
    if (root == NULL) { // 빈 노드이면 종료
        return;
    }
    inorder(root->left); // 왼쪽 서브트리를 먼저 방문
    printf("%d ", [REDACTED]); // 현재 노드 방문
    [REDACTED] // 오른쪽 서브트리 방문
}
```



Inorder

```
// 중위 순회: (왼쪽) → (현재 노드) → (오른쪽)
void inorder(struct Node *root) {
    if (root == NULL) { // 빈 노드이면 종료
        return;
    }
    inorder(root->left); // 왼쪽 서브트리를 먼저 방문
    printf("%d ", root->key); // 현재 노드 방문
    [REDACTED] // 오른쪽 서브트리 방문
}
```

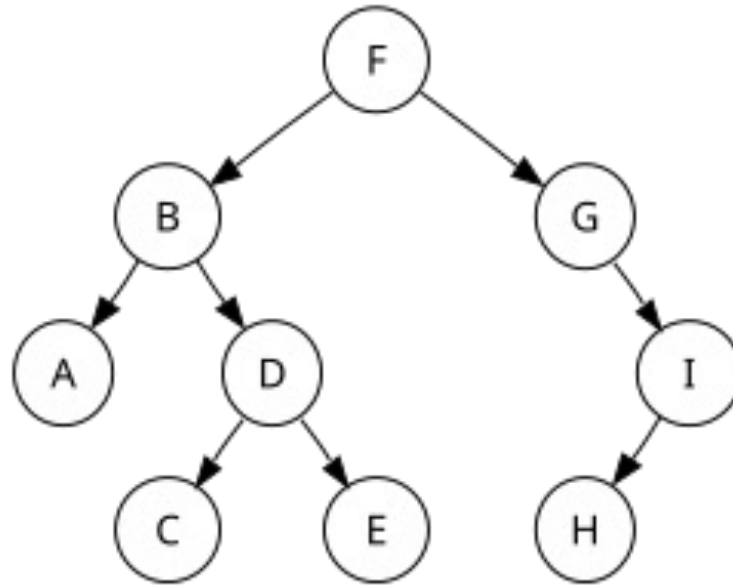


Inorder

```
// 중위 순회: (왼쪽) → (현재 노드) → (오른쪽)
void inorder(struct Node *root) {
    if (root == NULL) { // 빈 노드이면 종료
        return;
    }
    inorder(root->left); // 왼쪽 서브트리를 먼저 방문
    printf("%d ", root->key); // 현재 노드 방문
    inorder(root->right); // 오른쪽 서브트리 방문
}
```



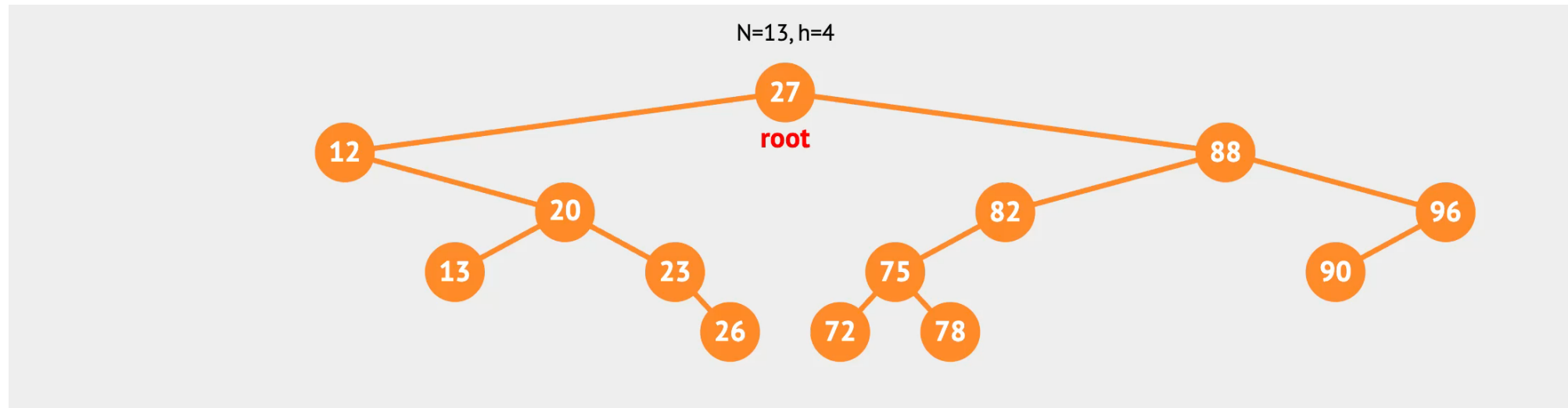
Postorder



left -> right -> root



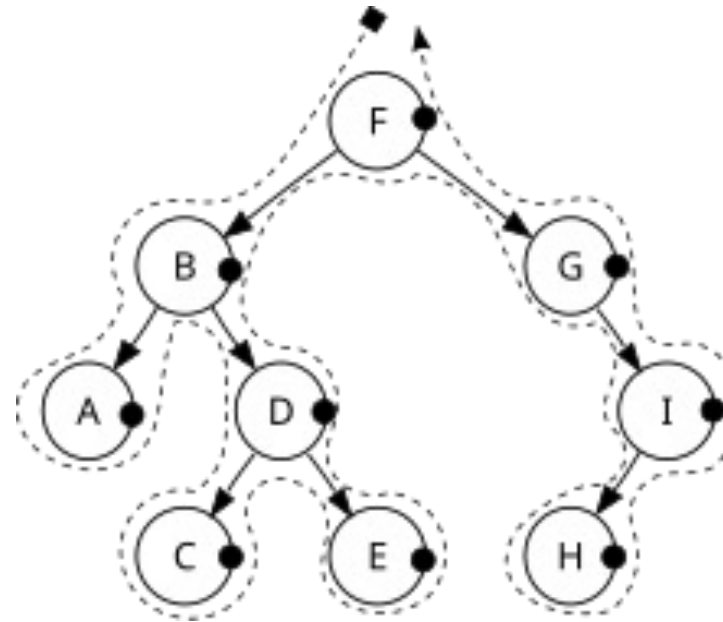
Postorder



left -> right -> root



Postorder



left -> right -> root



Postorder

```
// 후위 순회: (왼쪽) → (오른쪽) → (현재 노드)
void postorder(struct Node *root) {
    if (root == NULL) { // 빈 노드이면 종료
        return;
    }
    // 왼쪽 서브트리 방문
    // 오른쪽 서브트리 방문
    // 마지막에 현재 노드 방문
}
```



Postorder

```
// 후위 순회: (왼쪽) → (오른쪽) → (현재 노드)
void postorder(struct Node *root) {
    if (root == NULL) { // 빈 노드이면 종료
        return;
    }
    postorder(root->left); // 왼쪽 서브트리 방문
    // 오른쪽 서브트리 방문
    // 마지막에 현재 노드 방문
}
```



Postorder

```
// 후위 순회: (왼쪽) → (오른쪽) → (현재 노드)
void postorder(struct Node *root) {
    if (root == NULL) { // 빈 노드이면 종료
        return;
    }
    postorder(root->left); // 왼쪽 서브트리 방문
    postorder(root->right); // 오른쪽 서브트리 방문
    // 마지막에 현재 노드 방문
}
```



Postorder

```
// 후위 순회: (왼쪽) → (오른쪽) → (현재 노드)
void postorder(struct Node *root) {
    if (root == NULL) { // 빈 노드이면 종료
        return;
    }
    postorder(root->left); // 왼쪽 서브트리 방문
    postorder(root->right); // 오른쪽 서브트리 방문
    printf("%d ", root->key); // 마지막에 현재 노드 방문
}
```



Main.c 실행

```
int main(void) {
    int n;                // 입력받을 정수 개수를 저장할 변수
    struct Node *root = NULL; // BST의 루트 노드 (초기에는 비어 있음)

    printf("Enter number of integers: ");

    if (scanf("%d", &n) != 1) { // 정수 하나 입력받기 (개수)
        printf("Input error\n"); // 입력 실패 시 에러 출력
        return 1;                // 비정상 종료
    }

    printf("Enter %d integers: ", n);

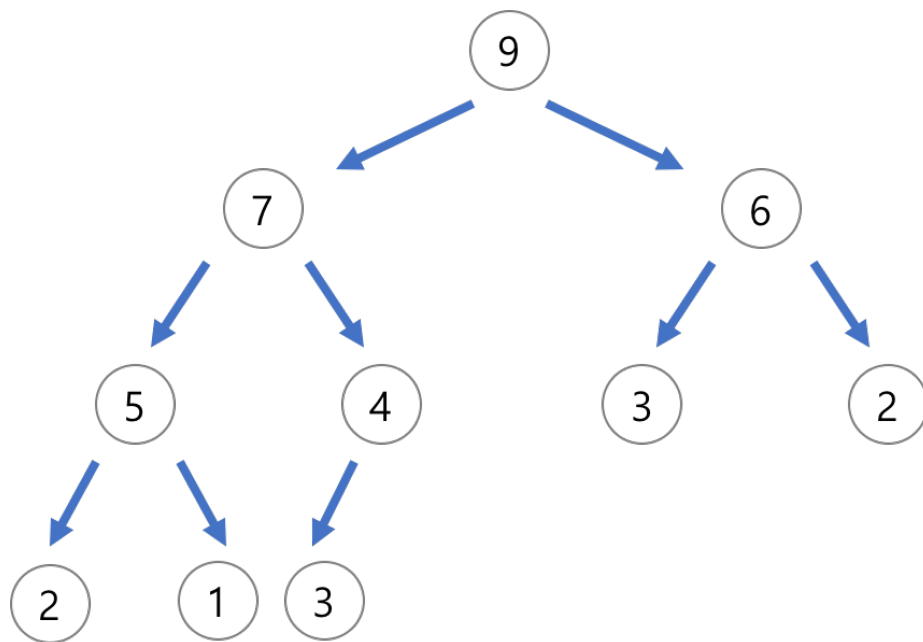
    for (int i = 0; i < n; i++) { // n개의 정수를 반복 입력
        int x;                    // 입력받을 정수를 저장
        if (scanf("%d", &x) != 1) { // 정수 입력 실패 검사
            printf("Input error\n");
            return 1;
        }
        root = insert(root, x); // 입력된 정수를 BST에 삽입
    }
}
```

```
(base) minseo@minseo-MacBookAir-2 1125-bst % ./run
13
27
12
20
13
23
26
88
82
75
72
78
96
90
Preorder: 27 12 20 13 23 26 88 82 75 72 78 96 90
Inorder: 12 13 20 23 26 27 72 75 78 82 88 90 96
Postorder: 13 26 23 20 12 72 78 75 82 90 96 88 27
```

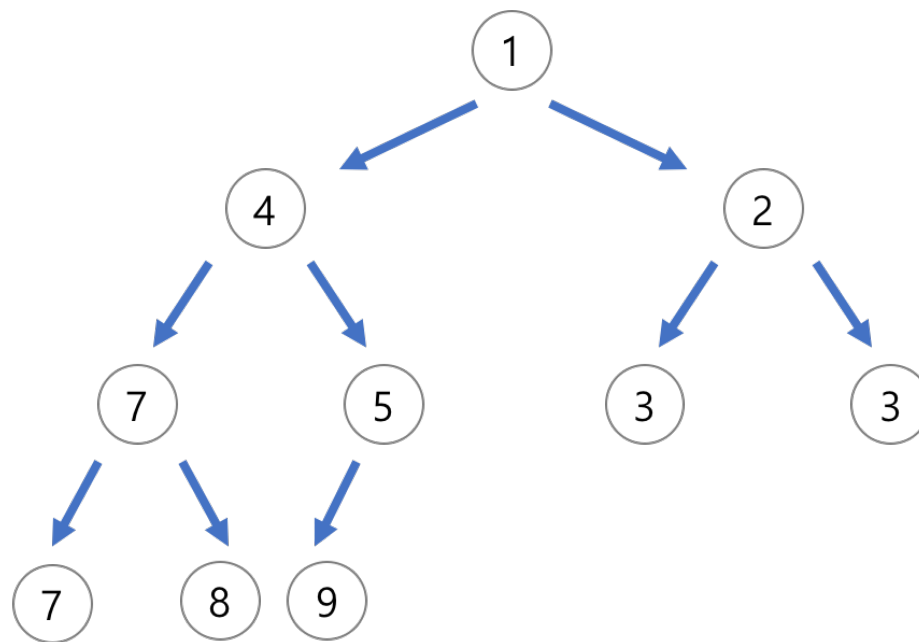
값들을 직접 바꿔가며 실행해 보세요



과제 안내



-최대 힙(max heap)-



-최소 힙(min heap)-



과제 안내

- Max Heap을 활용한 실생활 우선순위 시스템 구현
- <https://github.com/msforgood/data-structure-assist/tree/main/homework>



수고하셨습니다

자료구조 실습 11/12

EOF